

Topic 10: Strings in C++

9.1 What is a String?

- A string is a sequence of characters
- C-style string: array of char ending with null character `\0` — `char name[] = "Ahmad";`
- `std::string`: a class from `<string>` that manages character arrays automatically
- Use `std::string` in most cases — it is safer and easier than C-style strings

9.2 std::string Basics

```
#include <string>
string name = "Ahmad";
string greeting = "Hello, " + name;    // concatenation with +
cout << greeting << endl;
cout << "Length: " << name.length() << endl;
```

- `+` concatenates two strings
- Strings can be compared with `==`, `<`, `>`

9.3 Common string Methods

- `name.length()` or `name.size()` → number of characters
- `name.substr(1, 3)` → `"hma"` (start at 1, take 3 characters)
- `name.find("ma")` → 2 (starting index of "ma"), returns `string::npos` if not found
- `name.replace(0, 1, "O")` → replaces 1 character at index 0 with "O"
- `name.at(2)` → `'m'` (bounds-checked access, throws exception if out of range)
- `name.empty()` → true if string has no characters

9.4 Reading Strings

- `cin >> name;` reads one word
- `getline(cin, name);` reads entire line including spaces
- Always `cin.ignore()` between `cin >>` and `getline()` to discard the leftover `\n`

9.5 C-style String vs std::string

C-style <code>char[]</code>	<code>std::string</code>
Fixed size	Dynamic size
Manual null handling	Automatic
Faster (raw memory)	Safer and easier
<code>strcpy</code> , <code>strlen</code>	<code>.length()</code> , <code>+</code> operator

- Use `std::string` unless you are working at a low level or with legacy code
-